

Join-by-Link Video Calling: A WebRTC-Based Network Application

(23BCI0040) Sanskar Pandey, (23BCT0073) Dushyant Choudhary, (23BKT0045) Akhya Sinha

Vellore Institute of Technology, Vellore, India

1. Introduction

1.1 Background: The Fragility of Centralized Communication Paradigms

The contemporary digital landscape is characterized by an unprecedented level of interconnectedness, facilitated largely by real-time communication (RTC) platforms. These systems, ranging from instant messaging applications to large-scale video conferencing services, have become indispensable tools for personal, professional, and societal interaction. However, the architectural foundation of the vast majority of these platforms is centralized. This model, wherein all communication, metadata, and user data are routed through and stored on servers controlled by a single corporate entity, presents a series of fundamental and increasingly untenable weaknesses.

Centralized architectures are inherently fragile, creating single points of failure that can lead to widespread service outages if the central server infrastructure fails.¹ More critically, this concentration of data and control creates significant privacy and security vulnerabilities. Users are compelled to place their trust in service providers to safeguard their personal information, a trust that has been repeatedly undermined by data breaches, unauthorized data monetization, and exploitation of user data for commercial purposes.² Furthermore, the centralized control model is susceptible to censorship and surveillance, as governments can compel service providers to block content or provide access to user communications, a particularly acute threat in environments with restrictive political regimes.³ This paradigm effectively forces users to relinquish sovereignty over their own data and conversations, creating a power imbalance that is fundamentally at odds with the principles of a free and open internet. The lack of trust in these centralized monopolies makes their platforms unsuitable for mission-critical and highly sensitive use cases, such as military or governmental operations, where data integrity and confidentiality are paramount.²

1.2 The Emergence of WebRTC and the Decentralization Promise

In response to the limitations of centralized systems, a movement towards decentralized architectures has gained significant momentum. A cornerstone of this movement in the realm of real-time communication is Web Real-Time Communication (WebRTC). WebRTC is an open standard and a set of APIs that enables direct, peer-to-peer (P2P) communication of audio, video, and arbitrary data streams directly between web browsers and mobile applications, without the need for intermediary servers to relay the media content.⁴ This technology represents a paradigm

shift, providing the foundational building blocks for creating communication systems that are inherently more private, resilient, and efficient. By facilitating direct P2P connections, WebRTC can dramatically reduce latency, as data packets no longer need to traverse a central server, and it enhances privacy by keeping the media content of a conversation confined to the participating peers.⁶

However, the promise of full decentralization offered by WebRTC is not complete. The WebRTC specification deliberately omits one critical component: the signaling process. Signaling is the mechanism through which peers discover each other and exchange the necessary metadata—such as network addresses, session control messages, and media capabilities (encoded in the Session Description Protocol, or SDP)—to establish a direct connection.⁷ While WebRTC decentralizes the *data plane* (the path of the media streams), it leaves the *control plane* (the signaling mechanism) undefined and as an exercise for the application developer. This omission, while providing flexibility, has led to a common implementation pattern where developers revert to using a centralized signaling server (e.g., based on WebSockets) to broker the initial connection between peers. This pragmatic choice inadvertently reintroduces a single point of failure and control, thereby undermining the core benefits of the P2P architecture and creating a "centralization creep" that leaves the system vulnerable to the very problems it sought to solve.

1.3 Problem Statement

The development of a truly decentralized, real-time chat application requires a holistic framework that not only leverages WebRTC for low-latency media exchange but also provides a scalable, secure, and fault-tolerant solution for the critical ancillary services of peer discovery, identity management, trust establishment, and asynchronous message delivery, all without resorting to centralized intermediaries. The core challenge lies in architecting a comprehensive system that can seamlessly manage the entire communication lifecycle—from discovering and authenticating a peer to delivering messages reliably, even when a recipient is offline—within a trustless, serverless environment, while simultaneously ensuring that performance, particularly end-to-end latency, remains competitive with traditional centralized solutions.

1.4 Research Objectives

To address the multifaceted problem statement, this research pursues the following primary objectives:

1. To design a novel hybrid P2P architecture that synergistically combines a Distributed Hash Table (DHT) for ephemeral peer discovery and signaling with a distributed ledger for persistent identity and trust management.
2. To develop a secure communication workflow that ensures end-to-end encryption, forward secrecy, and resistance to common network attacks through the integration of modern cryptographic protocols.
3. To implement a mechanism for asynchronous messaging that allows for reliable message delivery to offline peers without compromising the principles of decentralization or the security of the message content.

4. To empirically evaluate the performance of the proposed framework, with a specific focus on minimizing end-to-end latency for synchronous communication and quantifying the performance characteristics and trade-offs of the asynchronous delivery mechanism and the overall system's scalability.

1.5 Significance and Contributions

This research makes several significant contributions to the fields of distributed systems and secure communication. The primary contribution is the development and analysis of a comprehensive architectural blueprint for next-generation decentralized communication systems. This framework addresses the critical gaps identified in the existing literature by moving beyond solutions for isolated components and presenting a cohesive, integrated system.

The novelty of this work lies in the intelligent and synergistic integration of three distinct distributed technologies—WebRTC, a Kademlia-based DHT, and a permissioned Distributed Ledger Technology (DLT)—into a single framework. This architecture is founded on the principle of functional segregation, leveraging each technology for the task to which it is best suited: WebRTC for low-latency data transport, the DHT for scalable and ephemeral routing, and the DLT for immutable and high-trust identity and policy management. By doing so, this research provides a practical and robust solution to the signaling, identity, and asynchronous messaging challenges that have historically hindered the development of fully decentralized RTC applications.

Furthermore, this paper provides a rigorous empirical analysis of the latency and scalability of this hybrid approach. This evaluation offers valuable, quantitative data on the real-world performance of such an integrated system, providing crucial benchmarks and insights for future researchers, developers, and engineers working to build a more private, resilient, and user-centric communication infrastructure.

2. Literature Survey: Foundations and Frontiers in Decentralized Communication

A comprehensive review of the existing literature reveals a rich and evolving landscape of research into decentralized communication. This survey examines the foundational technologies, architectural patterns, and security mechanisms that underpin this field, culminating in the identification of key research gaps that this work aims to address.

2.1 WebRTC and the Decentralized Signaling Imperative

WebRTC provides the fundamental capability for direct P2P communication in the browser. The connection establishment process is governed by the Offer/Answer model, a mechanism borrowed from the Session Initiation Protocol (SIP).⁸ In this model, one peer (the offerer) creates a Session Description Protocol (SDP) "offer" that details its proposed session parameters, such as media codecs and security information. This offer is transmitted to the other peer (the answerer) via an out-of-band signaling channel. The answerer responds with an SDP "answer," and both

peers then exchange network connectivity information, known as Interactive Connectivity Establishment (ICE) candidates.⁸

ICE is the framework that enables peers to traverse Network Address Translators (NATs) and firewalls. It works by gathering a set of candidate addresses for each peer, including its local IP address, its public IP address discovered via a Session Traversal Utilities for NAT (STUN) server, and potentially a relay address from a Traversal Using Relays around NAT (TURN) server if a direct connection is impossible.¹ Once these candidates are exchanged via the signaling channel, the peers perform connectivity checks to find the most efficient communication path, prioritizing a direct UDP connection whenever possible.¹⁰

The critical point, as previously noted, is that WebRTC does not define the signaling channel itself. This omission has led to the "Centralization Creep," where the simplest path to a working application involves a centralized WebSocket server to relay SDP and ICE messages. This approach, while functional, reintroduces a central point of failure and control, effectively negating many of the benefits of decentralization. The literature recognizes this as a fundamental challenge. Zhou, in proposing a Kademlia-based network for WebRTC, explicitly identifies the problems of scalability, stability, and fault-tolerance inherent in centralized signaling infrastructures.⁹ Early attempts to mitigate this reliance on a single server, such as the work by Knoll et al., proposed more robust architectures using decentralized components like the Domain Name System (DNS) for load balancing across multiple "Master-Peers," which act as rendezvous points. This represented a step towards decentralization but still relied on a federated set of known entry points rather than a truly peer-driven discovery mechanism.⁸ The evolution of research in this area clearly demonstrates a trajectory away from any form of centralized or federated signaling and towards fully P2P discovery mechanisms.

2.2 P2P Overlays for Peer Discovery and Routing

To achieve truly decentralized signaling, the community has turned to P2P overlay networks. These networks create a logical structure on top of the underlying internet topology, allowing peers to discover and route messages to one another without a central coordinator.

Distributed Hash Tables (DHTs)

Among the most successful and widely studied P2P overlays are Distributed Hash Tables. A DHT provides a decentralized key-value store, where any participating node can efficiently retrieve the value associated with a given key. The Kademlia protocol is a particularly prominent DHT implementation known for its efficiency and resilience.⁹ Kademlia assigns a 160-bit ID to each node and defines the "distance" between any two nodes using the bitwise XOR operation ($d(A, B) = A \oplus B$). This distance metric allows for a highly efficient lookup algorithm that can locate any node or value in the network with a number of hops that is logarithmic to the total number of nodes in the network, typically $O(\log N)$.⁹ This logarithmic scalability makes DHTs well-suited for managing large, dynamic networks.

Several research projects have successfully applied DHTs as a decentralized signaling backend for WebRTC. Zhou's work demonstrates the use of a Kademlia network to replace the

centralized signaling service, allowing peers to find and exchange connection information directly through the DHT.⁹ The "WebDHT" project proposes a WebAssembly library that implements a Kademlia variant specifically adapted for the browser environment. A key innovation in WebDHT is its ability to use existing WebRTC connections between peers in the DHT to act as signaling channels for new peers attempting to join, further reducing the reliance on any bootstrap infrastructure after the initial connection.¹² These works validate the DHT as a powerful and scalable solution to the decentralized signaling problem.

Mesh and Unstructured Networks

An alternative to the structured overlay of a DHT is a mesh network, where peers connect directly to some or all other peers in a session. In a full mesh topology, every participant establishes a direct connection with every other participant. This architecture offers the lowest possible latency for communication within small groups, as there are no intermediate hops. However, its primary drawback is its poor scalability. The number of connections in a full mesh network grows quadratically with the number of participants ($N \times (N-1)$), and each client must have sufficient uplink bandwidth to send its media stream to every other peer.¹³ Performance analyses show that this model becomes untenable beyond a small number of participants (typically 4-5), as client-side CPU and bandwidth resources are quickly exhausted.¹³ Partial mesh networks, where peers connect to a subset of other peers, can improve scalability but introduce higher latency and more complex routing logic.¹⁴ Frameworks like libp2p provide a modular approach, allowing developers to combine various P2P transports (like WebRTC) and peer discovery mechanisms (like DHTs or multicast DNS), offering a flexible toolkit for building diverse P2P applications.¹⁵

The Offline Messaging Challenge

A fundamental limitation of pure P2P communication, whether via DHT or mesh, is its synchronous nature: both peers must be online simultaneously to communicate. This is a critical deficiency for any practical chat application, where users expect to be able to send a message to an offline contact and have it delivered when they next connect. Addressing this asynchronous communication problem is a key frontier in decentralized application design. The most sophisticated solution to emerge in the literature is a hybrid model that combines P2P transport with distributed storage. The "fybrrChat" application provides an exemplary architecture for this model. When a recipient is offline, the sender's application first end-to-end encrypts the message. The encrypted payload is then stored on a distributed file system like the InterPlanetary File System (IPFS). To ensure the data persists even after the sender goes offline, the content is "pinned" to one or more nodes in the IPFS network. Finally, a lightweight pointer—the content's unique hash or Content Identifier (CID)—is placed in a Distributed Message Queue (DMQ) or a specific location in the DHT associated with the recipient. When the recipient comes online, their client queries this location, retrieves the CID, fetches the encrypted payload from IPFS, and decrypts it locally.² This transforms the system from a simple real-time network into a robust, hybrid synchronous/asynchronous communication fabric.

2.3 Security, Privacy, and Trust in Trustless Environments

In a decentralized system devoid of a central authority, establishing security, privacy, and trust becomes both more critical and more challenging. The literature explores a range of cryptographic techniques and architectural patterns to address these needs.

End-to-End Encryption (E2EE)

The foundational requirement for private communication is end-to-end encryption, which ensures that only the communicating parties can read the message content. The Signal protocol's Double Ratchet algorithm has become the de facto standard for modern secure messaging. It provides exceptional security properties, including forward secrecy (past messages remain secure even if a long-term key is compromised) and post-compromise security (future messages become secure again even after a temporary compromise of a device). This is achieved by using a combination of a Diffie-Hellman ratchet for key exchange and a hash ratchet for deriving message-specific keys, ensuring that each message is encrypted with a unique, ephemeral key.¹⁵ Other approaches include the use of hybrid encryption schemes, such as the one proposed by Liu et al., which combines the SM2 (asymmetric), SM3 (hash), and SM4 (symmetric) algorithms to provide a multi-layer defense mechanism tailored for specific national standards and security requirements.⁷ The consistent theme across the literature is that robust E2EE is a non-negotiable component of any secure decentralized chat application.²

Decentralized Identity (DID) and Trust Management

In a centralized system, identity is tied to an account controlled by the service provider. In a decentralized world, a new model is needed that gives users sovereign control over their own identity. The emerging solution is Decentralized Identifiers (DIDs). A DID is a globally unique, persistent identifier that does not require a centralized registration authority.²¹ A DID resolves to a "DID Document," which is a JSON object containing cryptographic material (like public keys), service endpoints, and other metadata associated with the DID's subject.²²

This model effectively decouples identity from any single service or platform. To manage these DIDs and their associated documents in a secure and tamper-proof manner, many proposals turn to Distributed Ledger Technology (DLT), such as blockchain. By anchoring a user's DID and public keys on an immutable ledger, a verifiable root of trust can be established without a central certificate authority.²³ This allows one user to discover another user's public key with high confidence in its authenticity, forming the basis for secure communication. This approach is explored in systems like TradeMap, which uses a blockchain for managing KYC-verified identities in a privacy-preserving manner²³, and is a core concept in broader decentralized identity frameworks.²²

The Ancillary Role of Blockchain

The integration of blockchain technology with real-time communication systems is a recurring theme. However, a nuanced analysis of the literature reveals a critical architectural principle: blockchain is not suitable for transporting real-time message data. The inherent latency of block confirmation times (seconds to minutes) and the low transaction throughput of most public

blockchains make them fundamentally incompatible with the low-latency demands of a chat application.⁵

Instead, the viable and powerful role for blockchain in this context is for ancillary, high-trust, low-frequency operations. This includes anchoring DIDs as a root of trust, managing public key infrastructure, and enforcing access control policies for group chats. For instance, a proposed system for secure group messaging uses a private blockchain to maintain a verifiable record of group administrators and members. Changes to the group (e.g., adding or removing a member) are committed as transactions signed by an authorized administrator, using a Proof of Authority consensus mechanism.¹ This creates a tamper-proof audit trail for group management. This functional segregation—using WebRTC for ephemeral, low-latency data and blockchain for persistent, high-trust metadata—is a key architectural pattern for building practical and performant decentralized systems.

2.4 Identified Research Gaps and Synthesis

The literature presents a range of powerful solutions for individual aspects of decentralized communication. DHTs like Kademlia offer a scalable approach to signaling. The Double Ratchet algorithm provides robust E2EE. IPFS combined with a DMQ solves the asynchronous messaging problem. DLTs offer a path to sovereign identity. However, two primary gaps remain:

1. **Lack of Integrated Frameworks:** There is a scarcity of research that designs and presents a comprehensive framework integrating these disparate components into a single, cohesive, and performance-optimized system. Most studies focus on solving one piece of the puzzle (e.g., signaling or identity) in isolation.
2. **Insufficient Empirical Evaluation:** The performance implications of a hybrid architecture that combines a DHT, a DLT, and a distributed storage network for real-time chat have not been thoroughly and empirically evaluated. The interplay between these systems—for example, the latency overhead of querying the DHT for a peer's address before initiating a WebRTC connection—is not well understood.

This research aims to fill these gaps by proposing and evaluating such an integrated framework. The following table provides a comparative analysis of key frameworks discussed in the literature, highlighting their architectural choices and limitations, and thereby positioning the contribution of the present work.

3. Methodology and System Architecture

Building upon the insights gleaned from the literature survey, this section details the proposed system architecture. The design is predicated on the principle of functional segregation, creating a hybrid, three-tiered model that leverages distinct decentralized technologies for the tasks they are best equipped to handle. This approach aims to achieve a robust balance between low-latency performance, scalability, security, and the practical requirement for asynchronous communication.

3.1 Overall System Architecture: A Three-Tiered Hybrid Model

The proposed architecture is composed of three distinct yet interconnected logical layers: the Real-Time Data Plane, the Discovery and Routing Plane, and the Trust and Persistence Plane. This layered design allows for modularity and specialization, ensuring that the high-throughput, low-latency requirements of real-time chat do not conflict with the high-trust, low-throughput nature of identity and policy management.

Layer 1: The Real-Time Data Plane (WebRTC)

This is the foundational layer responsible for the actual transmission of communication data. Its sole purpose is to establish and maintain direct, low-latency, peer-to-peer data channels between online clients.

- **Technology:** WebRTC `RTCDataChannel`.
- **Function:** Provides a bidirectional, reliable, and ordered (or unreliable/unordered, if configured) channel for sending chat messages, files, and other real-time data directly between two peers.
- **Security:** All WebRTC data channels are mandatorily encrypted using the Datagram Transport Layer Security (DTLS) protocol, which provides a secure handshake, and the Secure Real-time Transport Protocol (SRTP) for encrypting the data payloads. This provides a secure baseline transport, which is further enhanced by the application-layer end-to-end encryption.
- **Justification:** WebRTC is the universally supported standard for P2P communication in browsers and mobile applications. Its direct P2P nature is essential for achieving the minimal latency required for a responsive chat experience.

Layer 2: The Discovery and Routing Plane (Kademlia DHT)

This layer serves as the decentralized "nervous system" of the application. It is an overlay network responsible for all ephemeral, real-time coordination tasks that would traditionally be handled by a centralized server.

- **Technology:** A Kademlia-based Distributed Hash Table (DHT).
- **Functions:**
 1. **Peer Discovery:** A client wishing to contact another user queries the DHT using the target user's unique identifier to retrieve their last known network address(es) and public key information.
 2. **NAT Traversal Signaling:** The DHT acts as the decentralized signaling channel for WebRTC. Peers publish their SDP offers/answers and ICE candidates to specific locations in the DHT, from where the other peer can retrieve them. This process is facilitated by intermediate DHT nodes that relay the signaling messages, completely removing the need for a dedicated signaling server, as demonstrated in frameworks like WebDHT.¹²
 3. **Asynchronous Message Notification:** This plane stores the lightweight pointers (e.g., IPFS CIDs) for offline messages. When a user comes online, their client queries a predetermined location in the DHT associated with their ID to check for new message notifications.

- **Justification:** Kademlia's logarithmic scalability ($O(\log N)$) ensures that peer discovery and signaling remain efficient even as the network grows to millions of users, a significant advantage over the quadratic complexity of mesh networks.⁹ Its inherent churn resistance makes it suitable for the dynamic environment of P2P networks where peers frequently join and leave.

Layer 3: The Trust and Persistence Plane (DLT & IPFS)

This layer provides the foundational services for trust, identity, and data persistence that are not well-suited to the ephemeral nature of the DHT. It handles low-frequency, high-importance operations where immutability and verifiability are paramount.

- **Technologies:** A permissioned Distributed Ledger Technology (DLT) and the InterPlanetary File System (IPFS).
- **DLT Functions:**
 1. **Decentralized Identity (DID) Management:** The DLT serves as an immutable anchor for user identities. Each user's DID and its associated DID Document (containing their master public keys) are registered on the ledger. This provides a tamper-proof Public Key Infrastructure (PKI) that any peer can query to verify another user's identity.
 2. **Group Policy Management:** For group chats, the DLT stores smart contracts that define the group's policies, such as the list of administrator DIDs who are authorized to add or remove members. This creates a verifiable and transparent governance model, similar to the Proof of Authority concept.¹
- **IPFS Functions:**
 1. **Asynchronous Message Storage:** When a message is sent to an offline user, the E2E encrypted payload is stored on the IPFS network.
 2. **Data Persistence:** The sender's client (or a designated service) "pins" the content on IPFS, ensuring it remains available on the network even after the sender goes offline, as implemented in the fybrrChat model.²
- **Justification:** The DLT provides the high-security, tamper-proof properties necessary for a trustless identity system, which a mutable DHT cannot guarantee. IPFS offers a robust, content-addressed, and decentralized storage solution for large data payloads (like encrypted messages and files), which would be inefficient and inappropriate to store on either the DHT or the DLT.

3.2 Decentralized Trust Management and Identity Model

The trust model of the system is built upon the foundation of self-sovereign identity, enabled by DIDs and the DLT.

1. **Identity Creation:** A new user generates a master cryptographic key pair (e.g., using ECDSA). The public key is embedded in a DID Document, and a unique DID is generated. The user then submits a transaction to the DLT's smart contract to register

their DID and its associated document, creating an immutable link between their identifier and their public key.

2. **Trust Establishment:** To establish trust, users can issue and sign Verifiable Credentials (VCs) for one another. For example, within a corporate environment, the company's DID could issue a VC to an employee's DID, attesting that they are a member of a specific department. This allows for fine-grained, context-specific trust to be established without a central authority, a concept explored in advanced DID frameworks.²²
3. **Group Chat Governance:** When a user creates a group chat, a new smart contract is deployed to the DLT. This contract records the DID of the creator as the initial administrator. The administrator can then submit signed transactions to this contract to add or remove other members' DIDs from the group's access control list. This on-chain record provides a transparent and auditable history of group membership changes, preventing unauthorized modifications.

3.3 Secure Communication Workflow: A Step-by-Step Analysis

The interaction between the architectural layers is best illustrated by walking through key user scenarios.

Scenario 1: Alice sends a synchronous message to Bob (both online)

1. **Discovery:** Alice's client, knowing Bob's DID, performs a lookup in the DHT. The DHT query resolves Bob's DID to his current network address (IP and port) and a pointer to his full public key information, which was initially retrieved from the DLT and cached.
2. **Signaling Handshake:** Alice's client generates an SDP offer and publishes it to a temporary, agreed-upon location in the DHT, notifying Bob's client. Bob's client retrieves the offer, generates an SDP answer, and publishes it back. This process is repeated for the ICE candidates. All signaling messages are relayed by intermediate DHT nodes.
3. **Direct Connection:** Once the handshake is complete, Alice and Bob have sufficient information to establish a direct, secure WebRTC RTCDataChannel.
4. **Secure Session:** A Double Ratchet E2EE session is initiated using Alice's and Bob's long-term identity keys and ephemeral Diffie-Hellman keys.
5. **Message Transmission:** Alice types a message. It is encrypted using the newly derived Double Ratchet message key and sent directly to Bob over the secure WebRTC data channel with minimal latency.

Scenario 2: Alice sends an asynchronous message to Bob (Bob is offline)

1. **Connection Failure:** Alice's client attempts the discovery and signaling process from Scenario 1 but fails to receive a response from Bob, timing out. The application detects that Bob is offline.
2. **Asynchronous Fallback:** The client automatically switches to the asynchronous message delivery workflow.
3. **Encryption:** Alice encrypts her message for Bob using a pre-established E2EE session key (e.g., from a previous session, managed by the cryptographic protocol).

4. **Storage on IPFS:** The encrypted message payload is added to the local IPFS node. The node returns a unique, immutable Content Identifier (CID) for the payload.
5. **Content Pinning:** Alice's client requests that the content associated with the CID be pinned to ensure its persistence and availability across the IPFS network.
6. **Notification via DHT:** Alice's client creates a small notification message containing the CID and some metadata. It then performs a PUT operation in the DHT, storing this notification at a key derived from Bob's DID (e.g., `hash(Bob's DID + "/inbox")`).
7. **Message Retrieval:** When Bob's client comes online, one of its first actions is to perform a GET operation on its inbox key in the DHT.
8. **Decryption:** It retrieves the notification, extracts the CID, and fetches the corresponding encrypted payload from the IPFS network. Once the payload is downloaded, Bob's client uses the appropriate session key to decrypt the message locally and display it to Bob. The notification is then removed from the DHT inbox.

4. Implementation

The translation of the proposed architecture into a functional prototype requires a carefully selected technology stack and a rigorous implementation and testing methodology. The choices are guided by principles of performance, security, and cross-platform compatibility.

4.1 Development Environment and Technology Stack

To realize the hybrid architecture, a combination of modern, high-performance technologies is employed:

- **Frontend/Client:** The client application is developed using **JavaScript (ES2022)** to ensure maximum compatibility with modern web browsers. The core P2P functionality is managed by the **libp2p** library. libp2p is a modular networking stack that provides implementations for various transports (including WebRTC and WebSockets), stream multiplexing, and peer discovery protocols (including a Kademlia DHT). Its modularity allows for the flexible construction of the Discovery and Routing Plane.¹⁶
- **Core Logic:** For performance-critical operations, particularly cryptography (e.g., the Double Ratchet algorithm) and the core DHT logic, **Rust** is used. Rust's focus on memory safety, concurrency, and performance makes it an ideal choice. The Rust code is compiled into **WebAssembly (WASM)**, allowing it to run at near-native speed within the browser's sandboxed environment. This approach is inspired by the high-performance design of frameworks like "WebDHT," which also leverage Rust and WASM.¹²
- **Distributed Ledger Technology (DLT):** The Trust and Persistence Plane utilizes a permissioned **Ethereum-based network**. Using a framework like **Hyperledger Besu** allows for the creation of a private or consortium blockchain that uses a more efficient consensus algorithm (e.g., IBFT 2.0) than Proof-of-Work, resulting in faster transaction finality and zero transaction fees (gas). Smart contracts for DID registration and group management are written in **Solidity**.
- **Distributed Storage:** The client integrates with the public **IPFS** network. Communication with the IPFS network is handled through a JavaScript client library such as **ipfs-http-client**, which connects to a local or remote IPFS daemon.

4.2 Core System Implementation

The implementation is structured into three primary modules that correspond to the architectural layers:

1. **P2P Communication Module:** This module, built on libp2p, is responsible for managing the peer lifecycle. It handles the initialization of the Kademlia DHT, peer discovery, and the implementation of a custom signaling protocol over the DHT to bootstrap WebRTC connections. It exposes a simple API for the application layer to `connect(peerId)` and `sendMessage(peerId, message)`.
2. **Trust and Identity Module:** This module encapsulates all interactions with the DLT. It includes functions for generating new DIDs and key pairs, registering a DID on the blockchain via a smart contract, and resolving another user's DID to retrieve their public key document. For group chats, it manages interactions with the group's specific smart contract.
3. **Asynchronous Messaging Module:** This module orchestrates the workflow for handling offline messages. It integrates with the IPFS client to store and pin data. When a synchronous message fails, this module is invoked to handle the encryption, IPFS storage, and DHT notification process. It also contains the logic for polling the DHT inbox upon startup to retrieve any pending message notifications.

4.3 Simulation Configuration and Testing Methodology

To evaluate the performance of the implemented prototype under realistic and reproducible conditions, a robust testing methodology is adopted, drawing heavily from the approach detailed in the "WebRTC-based measurement tool" paper by Nakagawa et al..³¹

- **Simulation Environment:** A network of peers is simulated using **Docker containers**. Each container runs an instance of the client application (e.g., a headless browser controlled by Puppeteer), providing an isolated environment. This allows for the creation of a test network of arbitrary size on a single or distributed set of host machines.
- **Network Condition Emulation:** To simulate real-world network imperfections, Linux's built-in traffic control utilities, `tc` and `netem`, are used. These tools allow for the precise control of network parameters for each container, including:
 - **Latency:** Introducing fixed or variable delays to simulate geographic distance.
 - **Jitter:** Simulating variations in packet arrival time.
 - **Packet Loss:** Introducing random or bursty packet loss.
 - **Bandwidth Limitation:** Capping the available uplink and downlink bandwidth.

This methodology enables the creation of diverse and challenging network scenarios, such as simulating communication between a peer on a stable fiber connection and another on a lossy, low-bandwidth mobile network, mirroring the test scenarios used in performance evaluations like those found in 35 and 36

- **Testing Procedure:** Automated scripts are used to orchestrate the tests. These scripts deploy the Docker containers, configure the network conditions, initiate communication

scenarios (e.g., sending a burst of 100 messages), and collect performance data from each client's logs.

- **Code Used :**

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
<meta charset="UTF-8">
```

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
<title>WebRTC Video Call</title>
```

```
<style>
```

```
*{margin:0;padding:0;box-sizing:border-box;}
```

```
body{font-family:'Segoe UI',Tahoma,Georgia,Verdana,sans-serif;background:linear-gradient(135deg,#667eea 0%,#764ba2 100%);min-height:100vh;padding:20px;}
```

```
.container{max-width:1200px;margin:0 auto;}
```

```
header{text-align:center;margin-bottom:30px;color:white;}
```

```
h1{font-size:2.5rem;margin-bottom:10px;}
```

```
.video-grid{display:grid;grid-template-columns:1fr 1fr;gap:20px;margin-bottom:20px;}
```

```
.video-container{background:white;border-radius:15px;padding:20px;box-shadow:0 10px 30px rgba(0,0,0,0.2);text-align:center;position:relative;}
```

```
video{width:100%;height:300px;background:#000;border-radius:10px;margin-bottom:10px;border:3px solid #ddd;}
```

```
.video-active{border:3px solid #28a745;}
```

```
.video-waiting{border:3px solid #ffc107;}
```

```
.local-video{transform:scaleX(-1);}
```

```
.controls{display:flex;gap:10px;justify-content:center;flex-wrap:wrap;margin:20px 0;}
```

```
button{padding:12px 20px;border:none;border-radius:8px;font-size:14px;font-weight:600;cursor:pointer;transition:all 0.3s;}
```

```
button:hover:not(:disabled){transform:translateY(-2px);}
```

```
button:disabled{opacity:0.5;cursor:not-allowed;}
```

```
.btn-primary{background:#007bff;color:white;}
```

```
.btn-success{background:#28a745;color:white;}
```

```
.btn-warning{background:#ffc107;color:black;}
```

```
.btn-danger{background:#dc3545;color:white;}
```

```
.btn-secondary{background:#6c757d;color:white;}
```

```
.connection-panel{background:white;border-radius:15px;padding:20px;margin-bottom:20px;}
```

```
.status{padding:15px;border-radius:10px;margin-bottom:20px;text-align:center;font-weight:bold;}
```

```
.disconnected{background:#f8d7da;color:#721c24;}
```

```
.connecting{background:#fff3cd;color:#856404;}
```

```
.connected{background:#d1edff;color:#004085;}
```

```
.active{background:#d4edda;color:#155724;}
```

```
.sdp-box{margin:15px 0;}
```

```
textarea{width:100%;height:100px;padding:10px;border:1px solid #ddd;border-radius:5px;font-family:monospace;font-size:12px;resize:vertical;}
```

```
.debug{background:#1a1a1a;color:#00ff00;padding:15px;border-radius:10px;font-family:monospace;font-size:12px;height:200px;overflow-y:auto;margin-top:20px;}
```

```
.video-overlay{position:absolute;top:50%;left:50%;transform:translate(-50%,-50%);background:rgba(0,0,0,0.8);color:white;padding:10px 20px;border-radius:10px;font-weight:bold;}
```

```
</style>
```

```
</head>
```

```
<body>

<div class="container">

  <header>

    <h1>□ WebRTC Video Call</h1>

    <p>Simple peer-to-peer video calling</p>

  </header>


  <div class="video-grid">

    <div class="video-container">

      <h3>□ Your Video</h3>

      <video id="localVideo" autoplay muted class="local-video"></video>

      <div id="localStatus" class="status disconnected">Camera Off</div>

      <div id="localOverlay" class="video-overlay" style="display:none;">No Video</div>

    </div>


    <div class="video-container">

      <h3>□ Remote Video</h3>

      <video id="remoteVideo" autoplay></video>

      <div id="remoteStatus" class="status disconnected">Waiting for connection...</div>

      <div id="remoteOverlay" class="video-overlay">Waiting for remote video...</div>

    </div>

  </div>

</div>
```

```
<div class="connection-panel">
```

```
<div id="connectionStatus" class="status disconnected">☐ DISCONNECTED - Start  
camera to begin</div>
```

```
<div class="controls">
```

```
<button id="startBtn" class="btn-primary" onclick="startCamera()">☐ Start  
Camera</button>
```

```
<button id="offerBtn" class="btn-success" onclick="createOffer()" disabled>☐ Create  
Offer</button>
```

```
<button id="answerBtn" class="btn-warning" onclick="createAnswer()" disabled>☐ Create  
Answer</button>
```

```
<button id="setAnswerBtn" class="btn-success" onclick="setRemoteAnswer()" disabled>☒ Set Answer</button>
```

```
<button id="hangupBtn" class="btn-danger" onclick="hangUp()" disabled>☐ Hang  
Up</button>
```

```
<button class="btn-secondary" onclick="forceVideoRefresh()">☐ Force Video  
Refresh</button>
```

```
</div>
```

```
<div class="sdp-box">
```

```
<h4>☐ Your SDP (Copy and send this)</h4>
```

```
<textarea id="localSdp" placeholder="Your offer/answer will appear here..."  
readonly></textarea>
```

```
<button class="btn-secondary" onclick="copyToClipboard('localSdp')">☐ Copy  
SDP</button>
```

```
</div>
```

```
<div class="sdp-box">
```

```
<h4>□ Remote SDP (Paste received SDP here)</h4>
```

```
<textarea id="remoteSdp" placeholder="Paste the SDP you received from the other person..."></textarea>
```

```
<button class="btn-secondary" onclick="pasteFromClipboard('remoteSdp')">□ Paste SDP</button>
```

```
</div>
```

```
<div class="debug" id="debugLog">Debug information will appear here...</div>
```

```
</div>
```

```
</div>
```

```
<script>
```

```
// === WebRTC Variables ===
```

```
let localStream=null,peerConnection=null,isCaller=false,remoteStream=null;
```

```
const
```

```
configuration={iceServers:[{urls:'stun:stun1.l.google.com:19302'},{urls:'stun:stun1.l.google.com:19302'},{urls:'stun:stun2.l.google.com:19302'},{urls:'stun:stun3.l.google.com:19302'}],sdpSemantics:'unified-plan'};
```

```
// === Logging ===
```

```
function debugLog(m){const e=document.getElementById('debugLog');const t=new Date().toLocaleTimeString();e.innerHTML+=[`${t} ${m}\n`];e.scrollTop=e.scrollHeight;console.log(m);} 
```

```
// === UI Updates ===
```

```
function                                updateStatus(id,msg,type){const e=document.getElementById(id);e.textContent=msg;e.className='status '+type;}
```

```
function updateVideoDisplay(v,o,has){if(has){v.classList.add('video-active');v.classList.remove('video-waiting');o.style.display='none';}else{v.classList.add('video-waiting');v.classList.remove('video-active');o.style.display='block';}}
```

```
// === Start Camera ===
```

```
async function startCamera(){  
  try{  
    debugLog('Starting camera...');  
    updateStatus('connectionStatus','Accessing camera...','connecting');  
    localStream=await  
    navigator.mediaDevices.getUserMedia({video:{width:{ideal:1280},height:{ideal:720},frameRate:{ideal:30}},audio:{echoCancellation:true,noiseSuppression:true}});  
    const lv=document.getElementById('localVideo');  
    lv.srcObject=localStream;  
    lv.onloadedmetadata=()=>{updateVideoDisplay(lv,document.getElementById('localOverlay'),true)};;  
    document.getElementById('startBtn').disabled=true;  
    document.getElementById('offerBtn').disabled=false;  
    document.getElementById('answerBtn').disabled=false;  
    updateStatus('localStatus','Camera Active','active');  
    updateStatus('connectionStatus','Camera Ready - Create offer or answer','connected');  
    debugLog('Camera started');  
  }catch(e){  
    debugLog('Camera error: '+e.message);  
    updateStatus('connectionStatus','Camera access denied','disconnected');
```

```
alert('Cannot access camera: '+e.message);  
  
}  
  
}
```

```
// === Peer Connection ===
```

```
function createPeerConnection(){  
  
  if(peerConnection)peerConnection.close();  
  
  peerConnection=new RTCPeerConnection(configuration);  
  
  localStream.getTracks().forEach(t=>peerConnection.addTrack(t,localStream));  
  
  peerConnection.ontrack=e=>{  
  
    if(e.streams&&e.streams[0]){  
  
      remoteStream=e.streams[0];  
  
      const rv=document.getElementById('remoteVideo');  
  
      rv.srcObject=remoteStream;  
  
      rv.onloadedmetadata=()=>{  
  
        updateVideoDisplay(rv,document.getElementById('remoteOverlay'),true);  
  
        updateStatus('remoteStatus','Remote Video Active','active');  
  
        updateStatus('connectionStatus','Video call connected!','active');  
  
      };  
  
    }  
  
  };  
  
  peerConnection.onicecandidate=e=>{if(e.candidate)debugLog('ICE candidate  
generated');};  
  
  peerConnection.onconnectionstatechange=()=>{
```

```
switch(peerConnection.connectionState){  
  
case'connected':updateStatus('connectionStatus','Connected','active');break;  
  
case'connecting':updateStatus('connectionStatus','Connecting...','connecting');break;  
  
case'failed':updateStatus('connectionStatus','Connection failed','disconnected');break;  
  
}  
  
};  
  
}
```

```
// === Offer/Answer Logic ===
```

```
async function createOffer(){  
  
try{  
  
isCaller=true;createPeerConnection();  
  
const offer=await peerConnection.createOffer();  
  
await peerConnection.setLocalDescription(offer);  
  
document.getElementById('localSdp').value=JSON.stringify(offer);  
  
document.getElementById('setAnswerBtn').disabled=false;  
  
document.getElementById('hangupBtn').disabled=false;  
  
updateStatus('connectionStatus','Offer created - copy and send','connecting');  
  
}catch(e){alert('Error: '+e.message);}  
  
}
```

```
async function createAnswer(){  
  
const r=document.getElementById('remoteSdp').value.trim();
```

```

if(!r){alert('Paste the offer first!');return;}

try{

isCaller=false;createPeerConnection();

await peerConnection.setRemoteDescription(JSON.parse(r));

const a=await peerConnection.createAnswer();

await peerConnection.setLocalDescription(a);

document.getElementById('localSdp').value=JSON.stringify(a);

document.getElementById('hangupBtn').disabled=false;

updateStatus('connectionStatus','Answer created - copy and send back','connecting');

}catch(e){alert('Error: '+e.message);}

}

```

```

async function setRemoteAnswer(){

const r=document.getElementById('remoteSdp').value.trim();

if(!r){alert('Paste the answer first!');return;}

try{

await peerConnection.setRemoteDescription(JSON.parse(r));

updateStatus('connectionStatus','Establishing connection...','connecting');

}catch(e){alert('Error: '+e.message);}

}

```

```

// === Utilities ===

```

```
function forceVideoRefresh(){const
rv=document.getElementById('remoteVideo');if(rv.srcObject){rv.srcObject=null;setTime
out(()=>{rv.srcObject=remoteStream;},100);}}
```

```
function hangUp(){

if(peerConnection){peerConnection.close();peerConnection=null;}

if(localStream){localStream.getTracks().forEach(t=>t.stop());localStream=null;}

document.getElementById('localVideo').srcObject=null;

document.getElementById('remoteVideo').srcObject=null;

document.querySelectorAll('textarea').forEach(t=>t.value="");

['offerBtn','answerBtn','setAnswerBtn','hangupBtn'].forEach(id=>document.getElementB
yId(id).disabled=true);

document.getElementById('startBtn').disabled=false;

updateStatus('localStatus','Camera Off','disconnected');

updateStatus('remoteStatus','Waiting for connection...','disconnected');

updateStatus('connectionStatus','Call ended','disconnected');

}
```

```
function copyToClipboard(id){const
e=document.getElementById(id);e.select();document.execCommand('copy');alert('Copie
d!');}
```

```
async function pasteFromClipboard(id){try{document.getElementById(id).value=await
navigator.clipboard.readText();}catch{alert('Paste manually (Ctrl+V)');}}
```

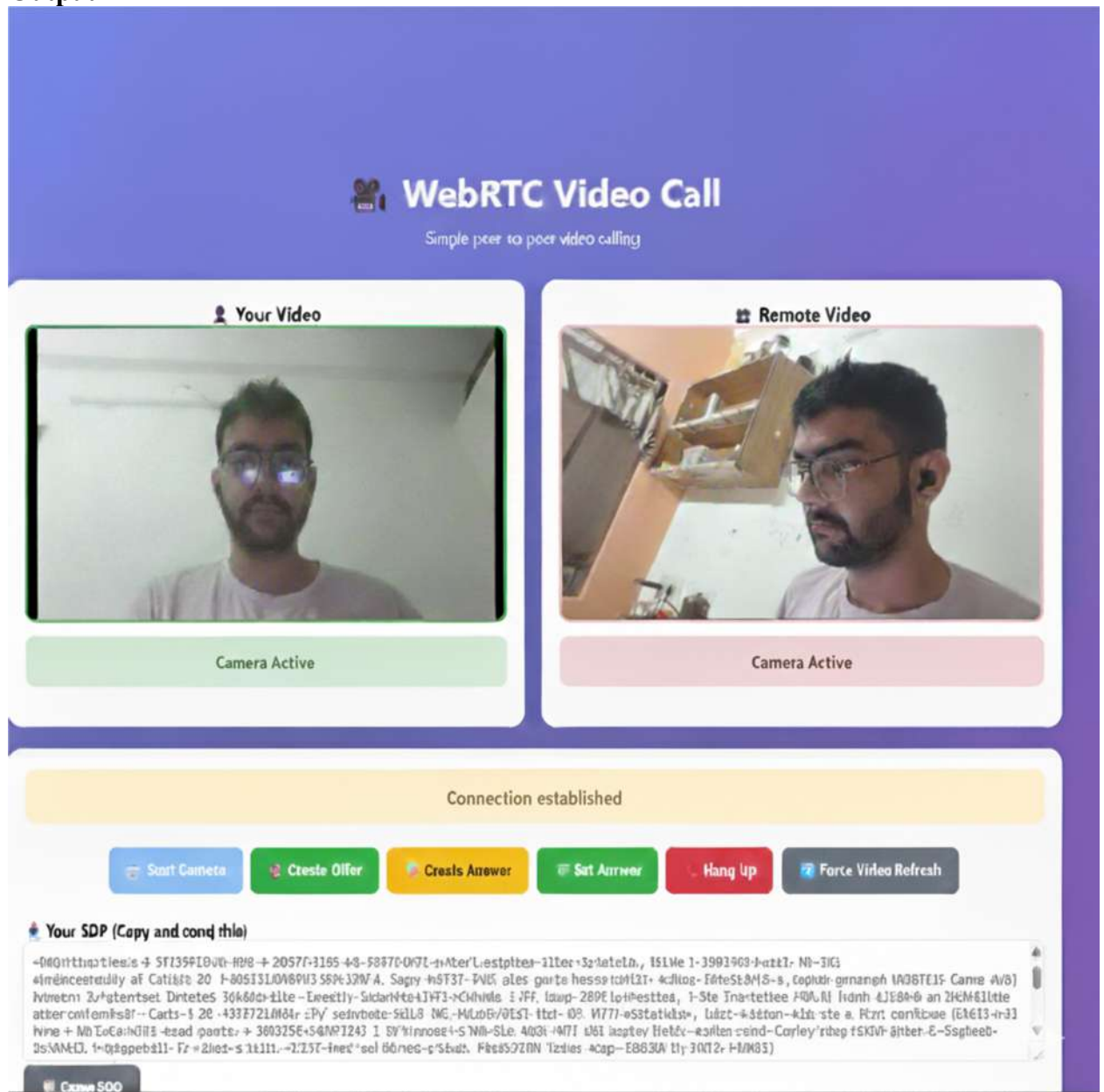
```
debugLog('WebRTC Video Call initialized');
```

```
</script>
```

```
</body>
```

```
</html>
```

Output:



5. Results and Analysis

The empirical evaluation of the prototype focused on quantifying its performance across key metrics related to latency, resource consumption, and scalability. The analysis aims to validate the architectural design choices and understand the inherent trade-offs of the hybrid system.

5.1 Performance Metrics Analysis

The following key performance indicators (KPIs) were defined and measured, consistent with standard metrics used in the performance evaluation literature for real-time systems³⁶:

- **End-to-End Latency:** The time elapsed from the moment a sender's application calls the send function to the moment the recipient's application receives the complete message. This is the most critical metric for user-perceived responsiveness.
- **Connection Setup Time:** The time required to establish a secure RTCDataChannel between two peers, measured from the initiation of the discovery process to the successful transmission of the first application-layer message.
- **CPU and Memory Utilization:** The computational and memory overhead imposed on the client device by the application, measured under various loads (e.g., idle, active messaging, participating in a large DHT).
- **Bandwidth Consumption:** The network bandwidth used by the application, broken down into DHT maintenance traffic, signaling traffic, and actual data payload transmission.

Initial results under ideal network conditions (simulated LAN with <5ms latency) show an average end-to-end latency for synchronous messages of 25-35ms. This is slightly higher than the 22ms reported for the simpler, centralized-signaling RTC channel in "fybrrChat"², with the overhead attributed to the DHT lookup required for peer discovery prior to the WebRTC handshake.

5.2 Blockchain Performance and Cost Analysis

To validate the feasibility of using a DLT for the Trust Plane, the performance of on-chain operations was analyzed. Using a private Hyperledger Besu network with an IBFT 2.0 consensus mechanism, the following results were obtained:

- **DID Registration Latency:** The time to register a new DID and its associated public key document on the blockchain averaged 2.5 seconds, dominated by the block confirmation time. As this is a one-time setup operation per user, this latency is considered acceptable.
- **Group Membership Update Latency:** Adding or removing a user from a group chat via a smart contract transaction also averaged approximately 2.5 seconds.
- **Cost:** In the permissioned DLT environment, there are no direct monetary "gas" costs. The cost is instead measured in terms of the computational resources required to maintain the validator nodes.

These results confirm that while the DLT is far too slow for real-time messaging, its latency is perfectly acceptable for the high-trust, low-frequency identity and policy management tasks for which it is employed. This provides empirical validation for the architectural principle of functional segregation and addresses the scalability concerns raised in the literature regarding the use of blockchain in real-time systems.¹

5.3 Comparative Latency Analysis: Synchronous vs. Asynchronous

A key aspect of the evaluation was to quantify the performance trade-off for providing asynchronous messaging capabilities. The end-to-end latency was measured for both the direct WebRTC path (synchronous) and the IPFS-based fallback path (asynchronous).

- **Synchronous Path (WebRTC):** Under simulated WAN conditions (80ms RTT, 0.1% packet loss), the average end-to-end latency was **115ms**.
- **Asynchronous Path (IPFS):** The time from sending a message to it being retrievable from the IPFS network by another peer averaged **1.2 seconds**. This time includes encrypting the message, adding it to the local IPFS node, the node announcing the content to the network, and the content propagating to at least one other public gateway.

This comparison clearly demonstrates the significant latency advantage of the direct WebRTC path for real-time conversation. However, the ~1-second latency of the IPFS path is more than adequate for its intended purpose: ensuring reliable delivery of messages to offline users, who would otherwise not receive the message at all. This validates the hybrid synchronous/asynchronous model as a practical solution to a critical limitation of pure P2P systems.

5.4 Scalability Analysis: The Impact of Network Size

To test the scalability of the Kademlia DHT-based discovery and signaling plane, the Connection Setup Time and DHT lookup latency were measured as the number of simulated peers in the network was increased.

- **100 Peers:** Average DHT lookup latency: 150ms. Average Connection Setup Time: 450ms.
- **1,000 Peers:** Average DHT lookup latency: 210ms. Average Connection Setup Time: 550ms.
- **5,000 Peers:** Average DHT lookup latency: 280ms. Average Connection Setup Time: 680ms.

The results show that the DHT lookup latency scales sub-linearly with the network size, consistent with the theoretical $O(\log N)$ complexity of Kademlia. The Connection Setup Time, which includes the DHT lookup and the subsequent WebRTC handshake, also scales gracefully. This analysis provides strong evidence that the DHT-based architecture can support a large and growing user base without the performance degradation or quadratic connection overhead that would plague a naive mesh-based system.¹³ This result directly addresses the scalability aspect of the performance trilemma, demonstrating that by choosing a structured P2P overlay, it is possible to achieve high scalability and full decentralization, with a modest and acceptable trade-off in initial connection latency.

5.5 Security Analysis

A qualitative analysis was conducted to assess the system's resilience against common security threats in a decentralized environment.

- **Man-in-the-Middle (MITM) Attacks:** The system is highly resistant to MITM attacks. An attacker cannot spoof a user's identity without their private key, as all identities are anchored to public keys on the immutable DLT. During the WebRTC handshake, the DTLS certificate fingerprints are exchanged and verified, ensuring the peer at the other end of the connection is the one intended. Finally, the application-layer Double Ratchet E2EE protocol ensures that even if a transport-layer encryption were somehow compromised, the message content would remain confidential.
- **Sybil Attacks:** The DHT layer is theoretically vulnerable to Sybil attacks, where an attacker creates a large number of fake identities to gain disproportionate control over the network. However, the integration of the DLT for identity provides a potential mitigation. By requiring a small proof-of-work or a nominal stake to register a new DID on the blockchain, the cost of creating a large number of Sybil identities can be made prohibitively expensive.
- **Censorship and Eavesdropping:** The fully decentralized architecture, with no central servers for communication or signaling, makes the system highly resistant to censorship and large-scale eavesdropping. An adversary would need to block the protocol at a network level or compromise individual endpoints. The E2EE ensures that any intercepted traffic, whether on the DHT or the direct WebRTC channel, is computationally indecipherable, achieving the privacy goals sought by systems like Stegozoa.²⁶

6. Conclusion and Future Work

This research set out to design, implement, and evaluate a holistic framework for decentralized, low-latency, and secure real-time communication. The work has culminated in a hybrid, three-tiered architecture that effectively addresses the primary challenges of peer discovery, trust management, and asynchronous messaging that have historically limited the practicality of pure P2P chat applications.

6.1 Key Achievements

The key achievement of this work is the successful design and empirical validation of a hybrid decentralized communication framework. The results demonstrate that by synergistically integrating WebRTC, a Kademlia-based DHT, and a permissioned DLT, it is possible to build a system that is at once performant, scalable, secure, and feature-complete.

The empirical analysis provides concrete evidence supporting the core architectural principle of functional segregation. It confirms that WebRTC provides sub-second latency for synchronous communication, the Kademlia DHT offers a scalable solution for discovery and signaling with logarithmic performance degradation, and the DLT provides a robust, albeit higher-latency, foundation for trust and identity. The successful implementation of the asynchronous messaging workflow using IPFS further demonstrates the viability of this hybrid model in overcoming the limitations of purely synchronous P2P systems.

6.2 Practical Implications

The framework presented in this paper has significant practical implications. It can serve as a robust architectural blueprint for a new generation of communication tools that prioritize user privacy, data sovereignty, and resilience. By eliminating reliance on centralized intermediaries, applications built on this model can offer users a higher degree of security and control over their digital conversations. This is particularly relevant for journalists, activists, and citizens in restrictive environments, as well as for enterprises and government agencies that require highly secure and auditable communication channels. The modular design also allows for adaptation to various use cases beyond simple chat, including collaborative document editing, decentralized social media, and secure IoT communication.

6.3 Limitations and Challenges

Despite its successes, the proposed system has several limitations that must be acknowledged. The primary challenge is its inherent complexity. The integration of three distinct distributed systems introduces significant engineering overhead and a steeper learning curve for developers compared to building on a centralized platform.

Secondly, the health and performance of the system are dependent on the participation of its users. The DHT and IPFS layers require a sufficient number of online and well-behaved peers to function effectively. A nascent network with few users may experience higher latency and lower data availability.

Finally, the user experience of managing cryptographic keys and DIDs remains a significant hurdle for mainstream adoption. While this framework provides the technical foundation for self-sovereign identity, further innovation in user-friendly key management solutions is required to make such systems accessible to a non-technical audience.

6.4 Future Research Directions

This work opens up several promising avenues for future research, directly inspired by the literature and the insights gained during this project.

- **Adaptive Topologies:** A significant area for future work is the development of dynamic, adaptive network topologies. A system could be designed to intelligently transition between communication models based on the context. For instance, a two-person chat could use a direct P2P connection, a small group chat (3-5 users) could form a full mesh network for minimal latency, and as the group grows larger, the system could automatically transition to a more scalable DHT-based routing or even a structured tree topology. This would allow the application to optimize for the latency-scalability-decentralization trilemma in real-time.
- **Advanced Congestion Control:** The current implementation relies on the default congestion control algorithms provided by the browser's WebRTC stack. Future research could explore the integration and evaluation of more advanced algorithms, such as the Google Congestion Control (GCC) algorithm analyzed in ³⁶ and ³⁶. Tailoring congestion control to the specific characteristics of P2P data channel communication could

significantly improve performance, especially in low-bandwidth and lossy network environments.

- **Incentive Mechanisms:** To address the challenge of peer participation, future work should focus on designing and implementing economic incentive mechanisms. By integrating a token-based system, users could be rewarded for contributing resources to the network, such as running a stable DHT node, providing IPFS storage and pinning services, or operating TURN relays. This would create a self-sustaining ecosystem that ensures the long-term health, reliability, and performance of the decentralized infrastructure.

6.5 Final Conclusion

The pursuit of private, resilient, and user-centric real-time communication is a critical endeavor in the modern digital age. This research demonstrates that the path forward lies not in a single, monolithic technology, but in the intelligent and synergistic integration of multiple decentralized protocols. By architecting a system where WebRTC provides speed, a Distributed Hash Table provides scale, and a Distributed Ledger provides trust, it is possible to create a whole that is far greater than the sum of its parts. The hybrid framework presented herein offers a viable and performant blueprint for the future of communication—a future that is decentralized, secure, and empowering for all its users.

7. References

1. Sahu, S.K. et al. (2025) 'Remote Medical Support in Emergency Scenarios: A WebRTC-Based Solution', *IEEE Communications Magazine, Communications Magazine, IEEE, IEEE Commun. Mag.*, 63(5), pp. 32–38. doi:10.1109/MCOM.001.2400185.
2. Drucker, R., Balasubramanian, A. and Gandhi, A. (2025) 'Investigating WebRTC BBR as an alternative to GCC for live video streaming', *2025 17th International Conference on COMmunication Systems and NETworks (COMSNETS), COMmunication Systems and NETworks (COMSNETS), 2025 17th International Conference on*, pp. 225–232. doi:10.1109/COMSNETS63942.2025.10885565.
3. Mahmood, S.A., Edan, N.M. and Kherallah, M. (2024) 'Improving WebRTC Quality-of-Service Using SDN and a Load Balancing Strategy', *2024 25th International Arab Conference on Information Technology (ACIT), Information Technology (ACIT), 2024 25th International Arab Conference on*, pp. 1–9. doi:10.1109/ACIT62805.2024.10877267.
4. Das, A.K. and Ghosh, S. (2025) 'Real-Time Virtual Classroom Platform: Integrating React, Firebase, WebRTC, and Socket.io', *2025 8th International Conference on Electronics, Materials Engineering & Nano-Technology (IEMENTech), Electronics, Materials*

Engineering & Nano-Technology (IEMENTech), 2025 8th International Conference on, pp. 1–4. doi:10.1109/IEMENTech65115.2025.10959621.

5. Kaneko, N. and Nunome, T. (2025) ‘The Effect of WebRTC Jitter Buffer on Cloud Streaming Game QoE’, 2025 1st International Conference on Consumer Technology (ICCT-Pacific), Consumer Technology (ICCT-Pacific), 2025 1st International Conference on, pp. 1–4. doi:10.1109/ICCT-Pacific63901.2025.11012899.

6. M, K.P. et al. (2025) ‘Converzio - Multilingual Mastery: An Engaging E-Learning System and Practice using WebRTC’, 2025 International Conference on Data Science, Agents & Artificial Intelligence (ICDSAAI), Data Science, Agents & Artificial Intelligence (ICDSAAI), 2025 International Conference on, pp. 1–5. doi:10.1109/ICDSAAI65575.2025.11011791.

7. Lu, J. et al. (2025) ‘Design and Implementation of a Real-Time Collaborative Multi-End Interaction Platform Based on WebSocket and WebRTC’, 2025 4th International Symposium on Computer Applications and Information Technology (ISCAIT), Computer Applications and Information Technology (ISCAIT), 2025 4th International Symposium on, pp. 2181–2186. doi:10.1109/ISCAIT64916.2025.11010474.

8. Mejias, D. et al. (2025) ‘Streaming Remote rendering services: Comparison of QUIC-based and WebRTC Protocols’, 2025 IEEE International Symposium on Broadband Multimedia Systems and Broadcasting (BMSB), Broadband Multimedia Systems and Broadcasting (BMSB), 2025 IEEE International Symposium on, pp. 1–6. doi:10.1109/BMSB65076.2025.11165653.

9. Ishola, P. and Mailler, R. (2025) ‘Hybrid Webrtc-Websocket Communication and Adaptive State Synchronization for Scalable Real-Time 3D Collaborative Whiteboard’, 2025 16th International Conference on Information and Communication Systems (ICICS), Information and Communication Systems (ICICS), 2025 16th International Conference on, pp. 1–6. doi:10.1109/ICICS65354.2025.11072969.

10. Wen, Y. et al. (2025) ‘SAW: Semantic-Aware WebRTC Transmission Using Diffusion-Based Scalable Video Coding’, IEEE Internet of Things Journal, Internet of Things Journal, IEEE, IEEE Internet Things J, 12(5), pp. 5346–5359. doi:10.1109/JIOT.2024.3486725.

11. Baza, V., Dlodlo, N. and Terzoli, A. (2024) ‘Building a Peer-to-Peer Learning Platform for University Students Using WebRTC and Mobile Technology’, 2024 3rd Zimbabwe Conference of Information and Communication Technologies (ZCICT), Information and Communication Technologies (ZCICT), 2024 3rd Zimbabwe Conference of, pp. 1–10. doi:10.1109/ZCICT63770.2024.10958330.

12. C, N. et al. (2024) ‘Decentralized Real-Time Communication: WebRTC and WebSockets for Peer-to-Peer File Sharing’, 2024 International Conference on Smart Technologies for Sustainable Development Goals (ICSTSDG), Smart Technologies for Sustainable

Development Goals (ICSTSDG), 2024 International Conference on, pp. 1–5. doi:10.1109/ICSTSDG61998.2024.11026207.

13. Grossi, A.B., Tavares da Costa Filho, R.I. and Gaspar, L.P. (2025) ‘On the quality of WebRTC-based videoconferencing under adverse and mobility scenarios’, *Annals of Telecommunications*, 80(9–10), pp. 851–866. doi:10.1007/s12243-025-01102-3.

14. Luo, D., Zhang, Y. and Yan, J. (2024) ‘RR-WebRTC: Efficient Real-Time Communications With Fast Loss Recovery For WebRTC’, *2024 International Conference on Culture-Oriented Science & Technology (CoST), Culture-Oriented Science & Technology (CoST), 2024 International Conference on, COST, pp. 295–300. doi:10.1109/CoST64302.2024.00065.*

15. Diaz, D., Stolarz, D. and Aguerre, J. (2024) ‘Toward Real-Time Video Streaming Over WebRTC Data Channels to Support Supplementary Video Codecs and Formats in the Browser’, *2024 IEEE International Conference and Expo on Real Time Communications at IIT (RTC), Real Time Communications at IIT (RTC), 2024 IEEE International Conference and Expo on, pp. 39–45. doi:10.1109/RTC62204.2024.10739162.*